Call Me Maybe

Cours 3 — Logits et selection de token (v2 — SDK corrige)

---

1. Ce que produit le modele a chaque etape

A chaque etape, le modele prend tous les tokens vus jusqu'ici et produit un **vecteur de logits** — un score par token du vocabulaire (~150 000 scores).

```
list[int] --> LLM --> list[float] (un float par token du vocab)
```

2. Le SDK — interface reelle

Le SDK simplifie tout : tu passes une **liste d'entiers**, tu recois une **liste de floats**. Pas de tenseurs a manipuler.

```
from llm_sdk import Small_LLM_Model
import numpy as np
model = Small_LLM_Model()   # charge Qwen3-0.6B
# encode() retourne un tenseur 2D -> convertir en list[int]
input_ids = model.encode('What is the sum of 2 and 3?')[0].tolist()
# -> [892, 318, 262, 4771, 286, 16, 290, 17, 30]
# get_logits_from_input_ids : list[int] -> list[float]
logits = model.get_logits_from_input_ids(input_ids)
# -> [0.23, -1.4, 8.2, ...]   ~150 000 floats
# Convertir en numpy pour le masquage
logits_np = np.array(logits)
```

IMPORTANT : get_logits_from_input_ids() fait deja la conversion interne. Tu ne manipules jamais de tenseurs PyTorch directement.

3. Des logits aux probabilites : le softmax

Les logits sont des scores bruts. Le **softmax** les transforme en probabilites entre 0 et 1 qui somment a 1. Un logit negatif ne signifie pas zero — juste moins probable.

Token	Logit	Prob.	Statut

`"fn_add_numbers"`	`8.2`	42%	Token le plus probable -> choisi par argmax
`"fn_greet"`	`4.6`	18%	Probable mais pas le meilleur
`"bonjour"`	`9.8`	??%	Score eleve MAIS invalide -> masque a -inf
`"{"`	`-1.2`	2%	Tres peu probable — logit negatif
`"autres"`	`...`	...	~149 995 autres tokens du vocabulaire

4. Le constrained decoding sur les logits

Tu interviens entre les logits et la selection. Tokens invalides -> logit a -inf -> probabilite 0 apres softmax.

Avant masquage	Apres masquage

`"fn_add_numbers" -> 8.2 [VALIDE]`
`"fn_greet" -> 4.6 [VALIDE]`
`"bonjour" -> 9.8 [invalide]`
`"{" -> 2.1 [invalide]`
`"42" -> -0.4 [invalide]`
	`"fn_add_numbers" -> 8.2 [VALIDE]`
	`"fn_greet" -> 4.6 [VALIDE]`
	`"bonjour" -> -inf [BLOQUE]`
	`"{" -> -inf [BLOQUE]`
	`"42" -> -inf [BLOQUE]`

5. Les 3 methodes de selection

Methode	Principe	Avantage	Inconvenient
Argmax	Toujours le token avec le score le plus eleve	Deterministe, 100% fiable	Pas de variete
Sampling	Tirage aleatoire selon les probabilites softmax	Sorties variees	Peut choisir des tokens improbables
Temperature	Divise les logits par T avant softmax	T<1 = plus sur ; T>1 = plus creatif	A regler soigneusement

```
import numpy as np
# 1. Argmax — deterministe (utilise dans le projet)
token_id = int(np.argmax(logits_np))
# 2. Softmax + sampling
def softmax(x):
    e = np.exp(x - np.max(x))   # stabilite numerique
    return e / e.sum()
probs    = softmax(logits_np)
token_id = np.random.choice(len(probs), p=probs)
# 3. Temperature
logits_temp = logits_np / 0.5   # T=0.5 -> plus deterministe
probs       = softmax(logits_temp)
```

6. La boucle de generation complete — avec le vrai SDK

```
# encode() retourne un tenseur 2D -> convertir en list[int]
input_ids = model.encode(prompt)[0].tolist()
while True:
    # 1. SDK : list[int] -> list[float] (pas de tenseur !)
    logits    = model.get_logits_from_input_ids(input_ids)
    logits_np = np.array(logits)
    # 2. TON TRAVAIL : tokens valides pour l'etat courant
    valid_ids = get_valid_tokens(state, vocab, ...)
    # 3. Liste vide = erreur critique
    if not valid_ids:
        raise ValueError('Aucun token valide')
    # 4. Masque : tous a -inf sauf les valides
    mask = np.full(len(logits_np), -np.inf)
    mask[valid_ids] = 0.0
    logits_np = logits_np + mask
    # 5. Selectionner le token suivant
    next_id    = int(np.argmax(logits_np))
    next_token = model.decode([next_id])
    # 6. Mettre a jour
    input_ids.append(next_id)
    generated_json += next_token
    state = transition(state, next_token, ctx)
    if next_id == model._tokenizer.eos_token_id:
        break
```

7. Points critiques

encode() retourne un tenseur 2D	model.encode(text) retourne tensor([[id1, id2, ...]]). Il faut faire [0].tolist() pour obtenir une list[int] utilisable.

get_logits : list[int] -> list[float]	Pas de tenseur en entree ni en sortie. Convertir avec np.array() pour utiliser numpy.
Liste vide = NaN apres softmax	Si get_valid_tokens() retourne [], tous les logits restent a -inf. Apres softmax : NaN. Detecter ce cas avant d'appliquer le masque.
Stabilite numerique	Toujours faire exp(x - max(x)) dans softmax. Sans ca, exp(8.2) sur 150 000 valeurs provoque des overflow.

Exercice

Q1. Logits : fn_add_numbers=8.2, fn_greet=4.6, bonjour=9.8, {=2.1. Sans constrained decoding, quel token est choisi ? Avec (seuls fn_add_numbers et fn_greet valides) ?

Q2. Pourquoi ajoute-t-on le masque aux logits (logits + mask) plutot que de supprimer les tokens invalides ?

Q3. Que se passe-t-il si get_valid_tokens() retourne une liste vide ? Comment le gerer ?

Reponses

Q 1	Sans contrainte : bonjour (9.8). Avec masque : fn_add_numbers (8.2 > 4.6, bonjour bloque a -inf).
Q 2	Le vecteur doit rester de taille fixe (150 000 valeurs). On met les invalides a -inf : apres softmax leur proba = 0. On ne supprime rien, on masque.
Q 3	Tous les logits a -inf -> softmax retourne NaN -> crash. Il faut detecter explicitement ce cas et lever une ValueError claire.

A retenir

- encode(text)[0].tolist() -> list[int] | get_logits_from_input_ids(list[int]) -> list[float].
- Convertir les logits en numpy : np.array(logits) avant tout masquage.
- Le masque s'additionne aux logits — le vecteur reste toujours de taille fixe.
- Liste vide de tokens valides = erreur a gerer explicitement.

Prochaine etape

Cours 4 — Le constrained decoding en profondeur : machine d'etats, get_valid_tokens() pour les 5 vraies fonctions du projet, gestion des types.